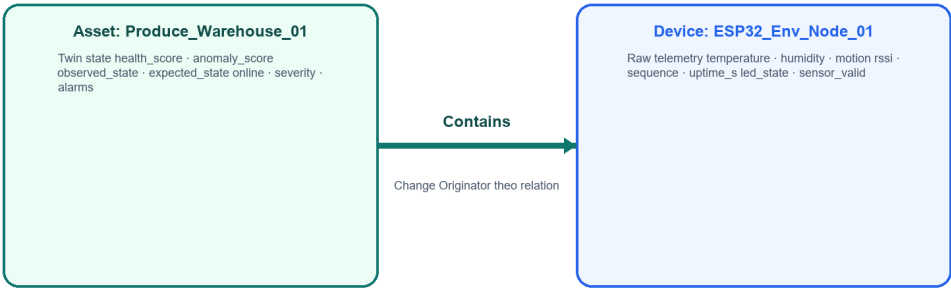


BÁO CÁO BÀI TẬP LỚN
MÔN INTERNET OF THINGS

XÂY DỰNG DIGITAL TWIN GIÁM SÁT
TRẠNG THÁI
VÀ PHÁT HIỆN BẤT THƯỜNG CHO
KHO BẢO QUẢN RAU QUẢ

MÔ HÌNH ENTITY VÀ RELATION



Mô hình trung tâm của đề tài

Hạng mục	Thông tin
Nền tảng	ThingsBoard Local 4.3.1.5 + PostgreSQL 18
Node IoT	ESP32 DevKit V1 · DHT22 · PIR · LED
Mô phỏng	Wokwi for VS Code
Năm thực hiện	2026

TÓM TẮT

Đề tài xây dựng một Digital Twin hai chiều cho kho bảo quản rau quả. Node ESP32 thu nhiệt độ, độ ẩm, chuyển động, RSSI, sequence, uptime và trạng thái LED; dữ liệu được gửi qua MQTT tới ThingsBoard Local. Rule Engine chuyển dữ liệu thô thành trạng thái twin cấp Asset, tính anomaly_score và health_score, vận hành máy trạng thái sáu mức, tạo/khôi phục alarm và đồng bộ trạng thái lệnh-phản hồi của LED.

Mô hình được triển khai tự động bằng script idempotent với 29 rule nodes, 35 connections và dashboard 19 widgets. Bộ acceptance test hiện có 140 lượt; TDR đạt 100.00%, FAR 0.00%, continuity 100.00% và p95 latency 9650 ms. Smoke test thời gian thật kéo dài 1.0 phút với chu kỳ 5.0 giây; kết quả lấy trực tiếp từ file máy đọc được.

Điểm khác biệt so với một dashboard IoT thông thường là hệ thống có mô hình entity/relation, có trạng thái kỳ vọng và quan sát, có lịch sử trạng thái, có logic phát hiện bất thường dựa trên ngưỡng, có vòng phản hồi RPC và có khả năng giải thích nguyên nhân alarm. Các ngưỡng sử dụng trong demo là cấu hình kiểm thử, không được xem là tiêu chuẩn bảo quản chung cho mọi loại nông sản.

Từ khóa: *Digital Twin, IoT, ThingsBoard, ESP32, MQTT, Rule Engine, anomaly detection, Wokwi.*

MỤC LỤC

TÓM TẮT	2
MỤC LỤC	2
DANH MỤC TỪ VIẾT TẮT	4
CHƯƠNG 1. TỔNG QUAN	5
1.1. Bối cảnh IoT và Digital Twin.....	5
1.2. Bài toán.....	5
1.3. Mục tiêu.....	5
1.4. Phạm vi và giới hạn.....	6
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ.....	7
2.1. Kiến trúc tổng thể	7
2.2. Thiết kế mạch	7
2.3. Giao thức và data flow.....	8
2.4. Entity model và relation	8
2.5. Data model.....	9
2.6. State machine.....	9

2.7. Mô hình bất thường	9
CHƯƠNG 3. TRIỂN KHAI	11
3.1. ThingsBoard Local và PostgreSQL	11
3.2. Triển khai idempotent bằng REST API	11
3.3. Firmware ESP32	11
3.4. Rule Engine	12
3.5. Dashboard	13
3.6. RPC và vòng phản hồi actuator	13
3.7. Bảo mật tối thiểu	14
CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ	15
4.1. Phương pháp	15
4.2. Kết quả acceptance	15
4.3. Smoke test vận hành liên tục	17
4.4. Kiểm thử Wokwi và RPC	17
4.5. Phân tích lỗi trong quá trình hoàn thiện	17
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	18
5.1. Kết quả đạt được	18
5.2. Hạn chế	18
5.3. Hướng phát triển	18
PHỤ LỤC A. HƯỚNG DẪN CHẠY NHANH	19
PHỤ LỤC B. CẤU TRÚC SOURCE	19
PHỤ LỤC C. TÀI LIỆU THAM KHẢO	19

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Diễn giải
DT	Digital Twin – bản sao số có trạng thái và vòng phản hồi
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
RPC	Remote Procedure Call
TDR	True Detection Rate
FAR	False Alarm Rate
RSSI	Received Signal Strength Indicator
CE/PE	Community Edition / Professional Edition

CHƯƠNG 1. TỔNG QUAN

1.1. Bối cảnh IoT và Digital Twin

IoT cung cấp khả năng quan sát đối tượng vật lý qua cảm biến và điều khiển cơ cấu chấp hành qua mạng. Tuy nhiên, một luồng telemetry đơn thuần chưa phải Digital Twin: hệ thống cần xác định đối tượng số đại diện cho cái gì, duy trì trạng thái theo thời gian, liên kết dữ liệu với cấu trúc tài sản, so sánh kỳ vọng với thực tế và phản hồi lại thế giới vật lý. Quan niệm này phù hợp với Digital Twin Model của ThingsBoard, trong đó entity, relation, telemetry, attributes, alarm và logic xử lý tạo thành mô hình có ngữ cảnh [1–3].

Trong bài toán bảo quản rau quả, nhiệt độ và độ ẩm bất thường có thể làm giảm chất lượng hàng hóa; mất dữ liệu, cảm biến kẹt hoặc mất gói cũng nguy hiểm vì dashboard có thể trông bình thường trong khi thiết bị đã hỏng. Do đó đề tài đồng thời theo dõi giá trị môi trường, chất lượng dữ liệu, chất lượng kết nối và độ khớp cơ cấu chấp hành.

1.2. Bài toán

Đối tượng vật lý được chọn là một zone kho bảo quản quy mô demo. ESP32 đọc DHT22 và PIR; LED đóng vai trò actuator/cảnh báo. Asset cấp kho là Digital Twin tổng hợp. Node cảm biến là Device con của Asset. Kiến trúc này học từ cách Smart Irrigation tổ chức Field Asset, sensor relation, alarm và command center, nhưng toàn bộ business logic đã được thay bằng nghiệp vụ kho bảo quản [4].

1.3. Mục tiêu

- Kết nối node ESP32/Wokwi tới ThingsBoard Local qua MQTT, chu kỳ 5 giây và tự reconnect.
- Mô hình hóa Asset, Device Profile, Device và relation Contains; tách raw telemetry khỏi twin state.
- Triển khai state machine sáu trạng thái và các luật bất thường định lượng, có score/severity/details.
- Xây dashboard realtime có lịch sử, alarm và RPC LED hai chiều.
- Đánh giá bằng ít nhất 10 lượt cho mỗi loại lỗi, latency, TDR, FAR, continuity và một smoke test vận hành liên tục.
- Bàn giao source, export, README, báo cáo, slide và storyboard video mà không lộ thông tin xác thực.

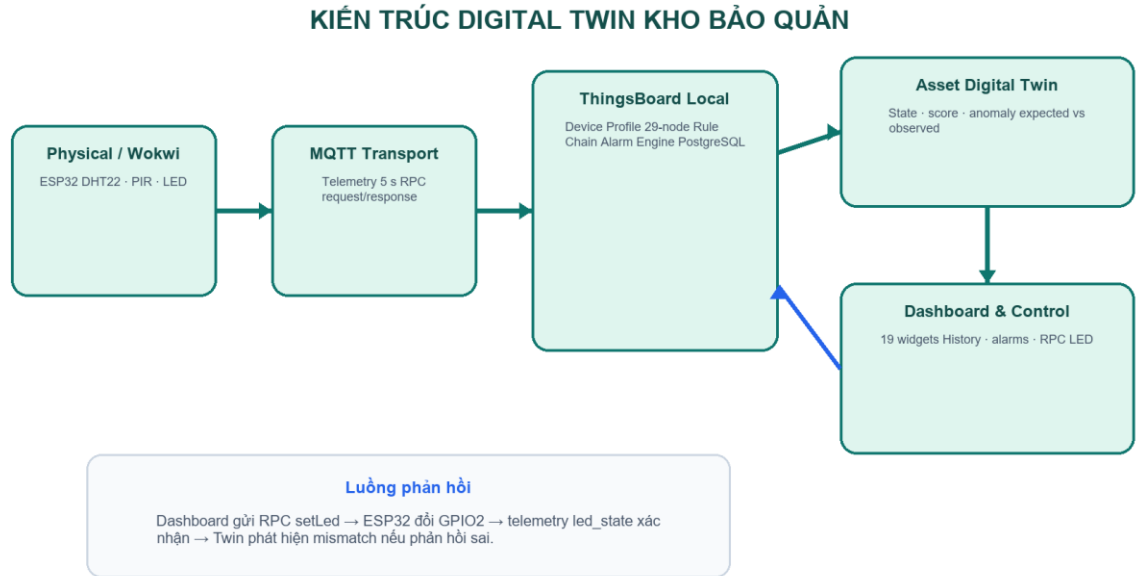
1.4. Phạm vi và giới hạn

Phiên bản chạy thực tế là ThingsBoard CE 4.3.1.5 vì môi trường không có license PE. Các khối Digital Twin được ánh xạ sang entity/relation, Rule Engine, time series, attributes và alarm tương thích; Calculated Fields/Solution Template đặc thù PE được thay bằng rule nodes. Mô hình hiện có một kho và một node; Wokwi chứng minh logic giao tiếp chứ không thay thế hiệu chuẩn cảm biến thật.

Trong phạm vi	Ngoài phạm vi
Nhiệt độ, độ ẩm, PIR, RSSI, sequence, uptime, LED	Mức nước, độ ẩm đất, bơm tưới
Một Asset kho và một Device ESP32	Tối ưu logistics nhiều kho
Luật ngưỡng/rate/stuck/connectivity	Mô hình ML dự báo hỏng hóc
Local Docker, Wokwi và RPC	Cloud production, TLS PKI hoàn chỉnh

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ

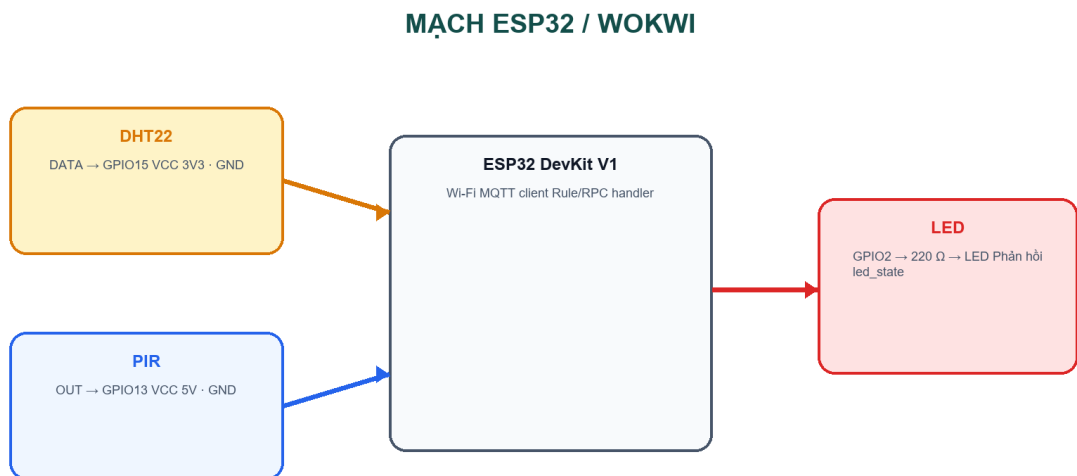
2.1. Kiến trúc tổng thể



Hình 2.1. Kiến trúc vật lý–giao tiếp–twin–dashboard

Luồng uplink bắt đầu tại ESP32, đi qua MQTT Device API vào Device. Rule Chain đọc ngưỡng và trạng thái trước, tính state/anomaly, lưu dữ liệu đã làm giàu trên Device, đổi originator theo relation để ghi twin state lên Asset, rồi tạo hoặc clear alarm. Luồng downlink bắt đầu từ widget RPC, ESP32 đổi LED và gửi lại `led\_state`; Rule Engine so sánh với `expected\_led\_state`.

2.2. Thiết kế mạch



Không có cảm biến mực nước/độ ẩm đất; mọi telemetry đều có nguồn từ mạch thật hoặc trạng thái giao tiếp.

Hình 2.2. Sơ đồ logic mạch và chân GPIO

Thành phần	Kết nối	Dữ liệu/trạng thái
DHT22	DATA GPIO15; 3V3; GND	temperature, humidity, sensor_valid
PIR	OUT GPIO13; 5V; GND	motion
LED + 220 Ω	GPIO2; GND	led_state / RPC setLed
Wi-Fi ESP32	Wokwi-GUEST hoặc LAN	rsssi, MQTT session

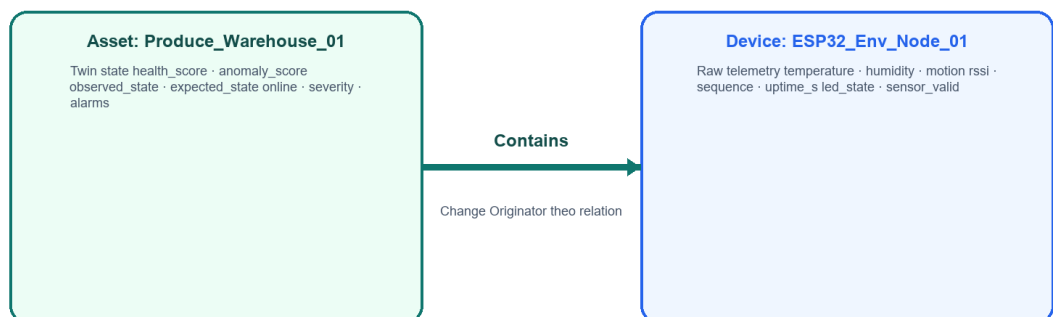
2.3. Giao thức và data flow

MQTT được chọn do overhead thấp, mô hình publish/subscribe và khả năng RPC qua topic. Token Device làm username; payload telemetry được publish vào `v1/devices/me/telemetry`. ESP32 subscribe `v1/devices/me/rpc/request/+` và trả lời ở topic response tương ứng [5–6]. Khi mất Wi-Fi/MQTT, firmware reconnect không chặn vòng lặp quá lâu và ThingsBoard phát lifecycle inactivity event [7].

```
{
  "temperature": 22.4, "humidity": 65.1, "motion": false,
  "rsssi": -52, "sequence": 1042, "uptime_s": 5210,
  "led_state": false, "sensor_valid": true,
  "firmware_version": "1.0.0"
}
```

2.4. Entity model và relation

MÔ HÌNH ENTITY VÀ RELATION



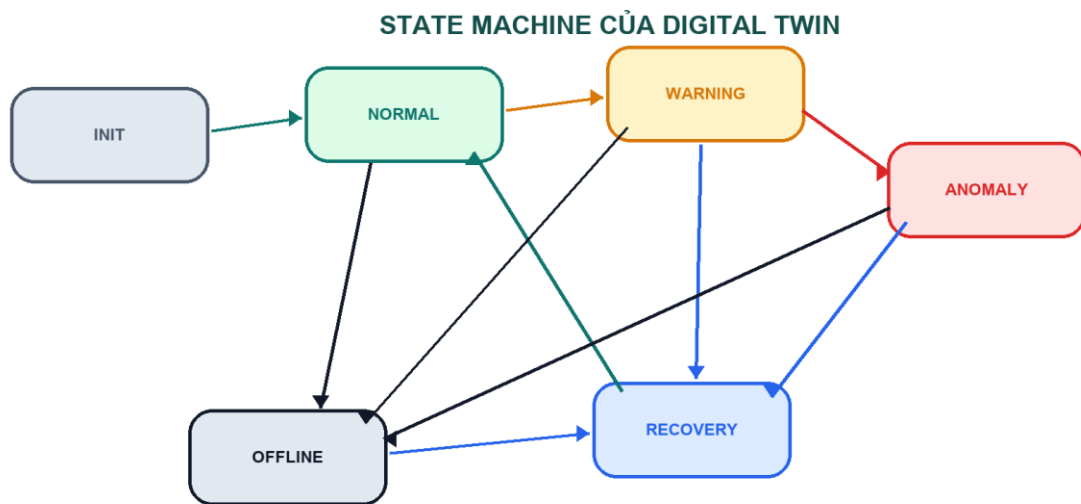
Hình 2.3. Asset–Device và nơi lưu dữ liệu

Asset `Produce\_Warehouse\_01` là identity ổn định của kho; Device `ESP32\_Env\_Node\_01` có thể được thay thế mà không làm mất identity kho. Relation `Contains` có hướng từ Asset tới Device. Change Originator truy vấn relation theo hướng TO từ Device để tìm Asset cha, giúp tái sử dụng Rule Chain khi mở rộng nhiều zone.

2.5. Data model

Lớp	Key tiêu biểu	Mục đích
Raw Device telemetry	temperature, humidity, motion, rssi, sequence, uptime_s, led_state	Bảng chứng gốc và chẩn đoán node
Device server attributes	thresholds, expected_state, security_mode, expected_led_state	Cấu hình không hard-code
Asset twin telemetry	observed_state, score, anomaly_type, last_seen, online	Lịch sử Digital Twin
Asset server attributes	state/score/anomaly mới nhất	Đọc nhanh và widget latest value
Alarm	type, severity, details, start/end/clear	Theo dõi vòng đời bất thường

2.6. State machine



OFFLINE ưu tiên cao nhất · RECOVERY cần 3 cửa sổ sạch · expected_state được so với observed_state

Hình 2.4. Máy trạng thái và các chuyển tiếp chính

OFFLINE có ưu tiên cao nhất vì không có dữ liệu mới để khẳng định môi trường an toàn. RECOVERY là trạng thái tạm, yêu cầu ba cửa sổ sạch để hạn chế dao động. WARNING dùng cho bất thường điểm thấp; ANOMALY dùng cho score từ 50 trở lên. `state\_match` là phép so sánh trực tiếp observed/expected để hỗ trợ vận hành theo mục tiêu.

2.7. Mô hình bất thường

Nhóm	Điều kiện demo	Để
		m

Nhóm	Điều kiện demo	Điểm
Nhiệt độ	<18 hoặc >26 cảnh báo; <15 hoặc >30 nghiêm trọng	40 / 90
Độ ẩm	<55 hoặc >75 cảnh báo; <45 hoặc >85 nghiêm trọng	40 / 90
Rate	$ dT/dt > 2$ °C/phút; $ dH/dt > 10$ %RH/phút	70 / 85
Sensor stuck	T và H đổi <0,01 trong 12 mẫu	70
Invalid sensor	Ba mẫu thiếu/ngoài miền	60
Sequence/restart	gap/trùng/giảm hoặc uptime giảm	45–80 / 65
Signal/motion	RSSI $\leq -80/-90$; ARMED và motion=true	35/80 · 70
Actuator mismatch	expected LED khác feedback 3 mẫu	75
Connection lost	INACTIVITY_EVENT	100

Khi nhiều điều kiện cùng đúng, luật có score lớn nhất trở thành `anomaly\_type`; danh sách đầy đủ vẫn nằm trong `active\_anomalies`. Công thức cuối cùng là `health\_score = max(0, 100 - anomaly\_score)`. Alarm details lưu state, score, last_seen và JSON giải thích detection để truy vết.

CHƯƠNG 3. TRIỂN KHAI

3.1. ThingsBoard Local và PostgreSQL

Hai service được quản lý bởi Docker Compose: PostgreSQL 18 lưu cấu hình/telemetry và `thingsboard/tb-node:4.3.1.5` mở Web UI 8080, MQTT 1883 và MQTTS 8883. Named volume giữ dữ liệu qua restart. State checker chạy chu kỳ 5 giây; inactivity timeout của Device là 5 giây để còn headroom xử lý dưới tiêu chí 15 giây.

```
docker compose up -d
docker compose ps
docker compose logs -f thingsboard-ce
```

3.2. Triển khai idempotent bằng REST API

`thingsboard/setup.py` dùng API key hoặc tài khoản Tenant Administrator để tìm theo tên, cập nhật nếu đã tồn tại và chỉ tạo mới khi thiếu. Script sinh entity/profile/relation, deploy metadata Rule Chain, tạo dashboard, đặt attributes, xuất JSON đã khử bí mật và sinh file runtime local. Cách này giảm lỗi thao tác UI và cho phép tái lập trên instance khác.

Artifact	Tên/định lượng
Asset Profile / Asset	Produce_Warehouse / Produce_Warehouse_01
Device Profile / Device	ESP32_Environment_Node / ESP32_Env_Node_01
Relation	Asset --Contains--> Device
Rule Chain	Produce Warehouse Digital Twin - Processing; 29 nodes; 35 connections
Dashboard	Produce Warehouse - Digital Twin; 19 widgets

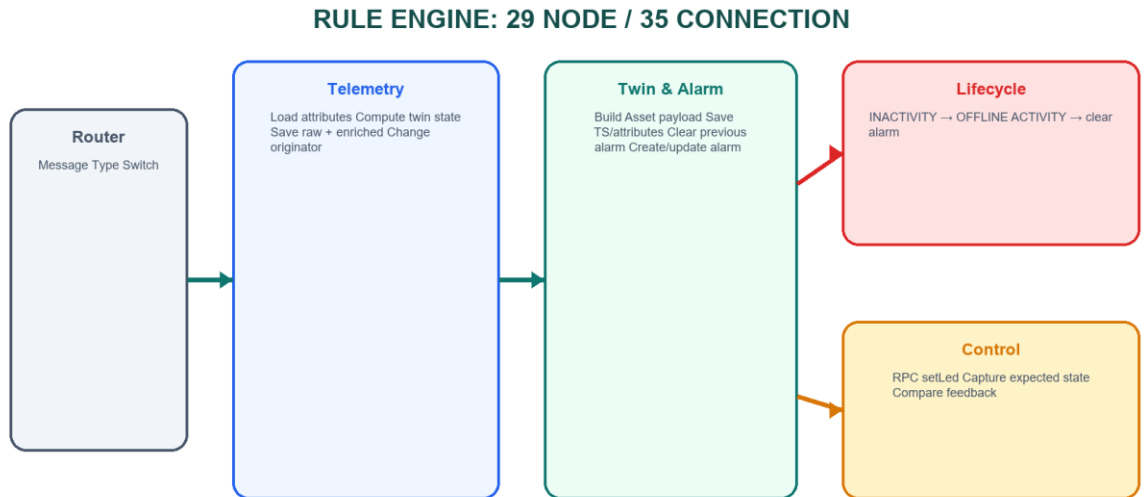
3.3. Firmware ESP32

Firmware khởi tạo cảm biến/chân GPIO, kết nối Wi-Fi và MQTT, đăng ký callback RPC, sau đó đọc và publish mỗi 5 giây. DHT lỗi không bị thay bằng số ngẫu nhiên: payload đánh dấu `sensor\_valid=false`. Sequence tăng đơn điệu; uptime lấy từ `millis()`. Fault mode chỉ phục vụ demo có kiểm soát và luôn có `clearFaults`.

```
loop():
  maintainWiFi(); maintainMqtt(); mqtt.loop();
  if (sampleDue) { readSensors(); applyFaultMode(); publishTelemetry(); }
  handleLedAndRpc();
```

Build PlatformIO đã hoàn tất cho board esp32dev. Tài nguyên build gần nhất: RAM xấp xỉ 13,7% và flash 58,1%. `firmware/secrets.h` được sinh local, bị git-ignore và không có trong package nộp.

3.4. Rule Engine

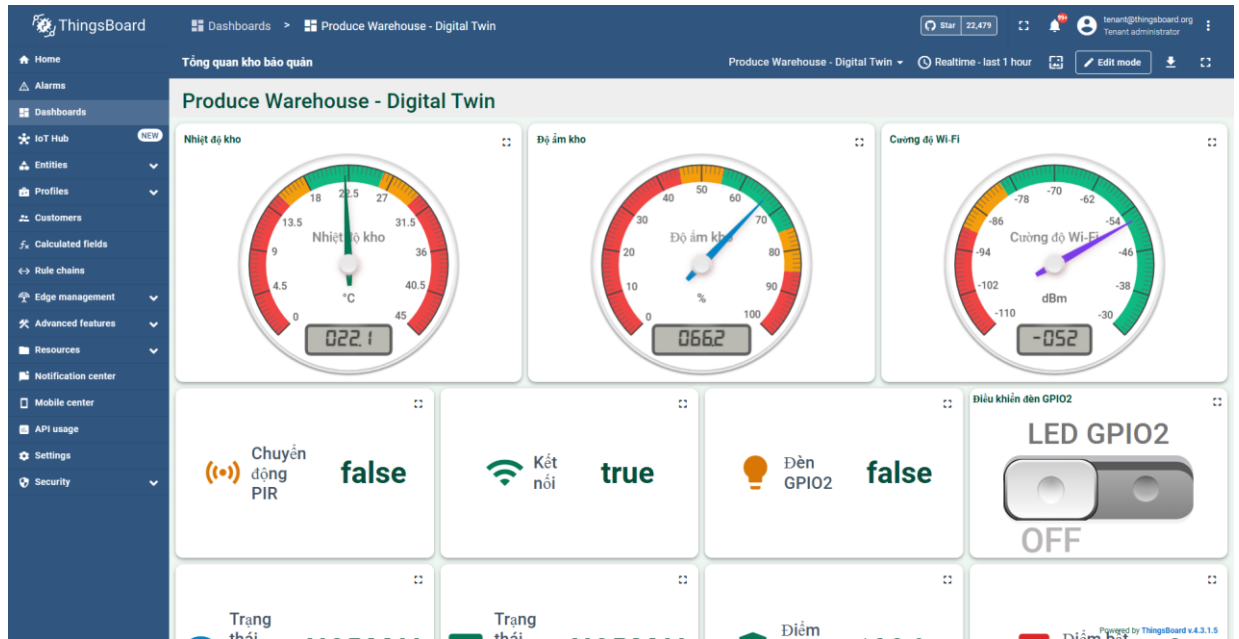


Hình 3.1. Các nhóm chức năng của Rule Chain

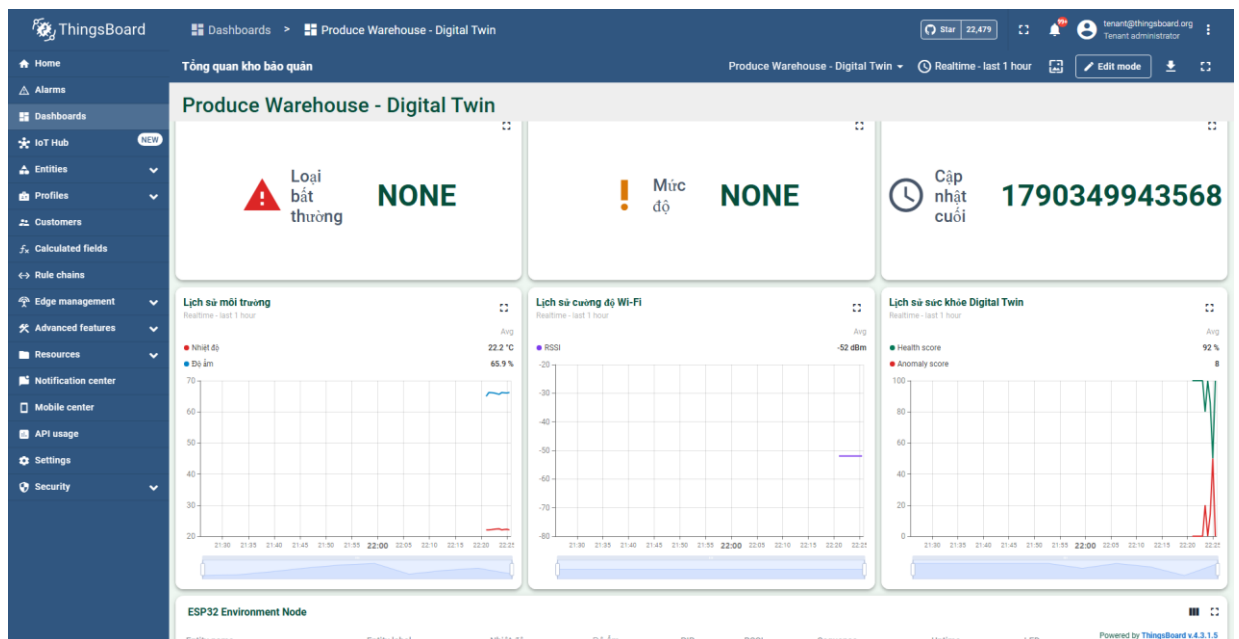
Nhánh telemetry enrich server attributes và latest telemetry trước khi chạy JavaScript. Kết quả được lưu song song trên Device và Asset. Nhánh lifecycle biến INACTIVITY_EVENT thành OFFLINE/CONNECTION_LOST; ACTIVITY_EVENT chỉ clear alarm kết nối để tránh ghi đè trạng thái telemetry thật. Nhánh RPC chuyển lệnh setLed thành expected state để phát hiện mismatch.

Node tạo alarm bật chế độ debug 'failures only', phù hợp hướng dẫn monitoring của ThingsBoard: chỉ lưu message lỗi, không ghi toàn bộ traffic. Trong quá trình kiểm thử, cơ chế này giúp phát hiện lỗi timestamp test ở tương lai; sau khi sửa, không còn failure mới tại node alarm [8].

3.5. Dashboard



Hình 3.2. Dashboard Digital Twin sau QA giao diện



Hình 3.3. Dữ liệu time-series của môi trường, kết nối và sức khỏe Twin

Dashboard được tổ chức theo bốn lớp: Environment (temperature/humidity/motion/RSSI), Digital Twin (state, score, health, match, online), History (biểu đồ thời gian) và Operations (alarm + RPC LED + fault actions). Màu xanh biểu thị bình thường, vàng cảnh báo, đỏ bất thường và xám offline. Dashboard không hiển thị mực nước hoặc độ ẩm đất vì mạch không có các cảm biến này.

3.6. RPC và vòng phản hồi actuator

```
{ "method": "setLed", "params": { "state": true } }
{ "method": "getStatus", "params": {} }
{ "method": "setFaultMode", "params": { "mode": "HIGH_TEMPERATURE" } }
```

Firmware chấp nhận boolean trực tiếp hoặc object có `state/value`, cập nhật GPIO2 và trả JSON trạng thái. Dashboard đọc telemetry `led\_state` thay vì giả định lệnh thành công. Nếu expected khác feedback trong ba mẫu, Twin tạo ACTUATOR_MISMATCH – đặc trưng cần thiết của một hệ thống hai chiều.

3.7. Bảo mật tối thiểu

- API key và token chỉ tồn tại trong `.env.local`/`secrets.h`, đều bị git-ignore.
- Export JSON và báo cáo không chứa credential; test không in token ra stdout.
- API key được truyền qua biến môi trường và nên thu hồi sau triển khai.
- Môi trường production cần đổi mật khẩu PostgreSQL, bật TLS MQTT và giới hạn cổng theo firewall.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ

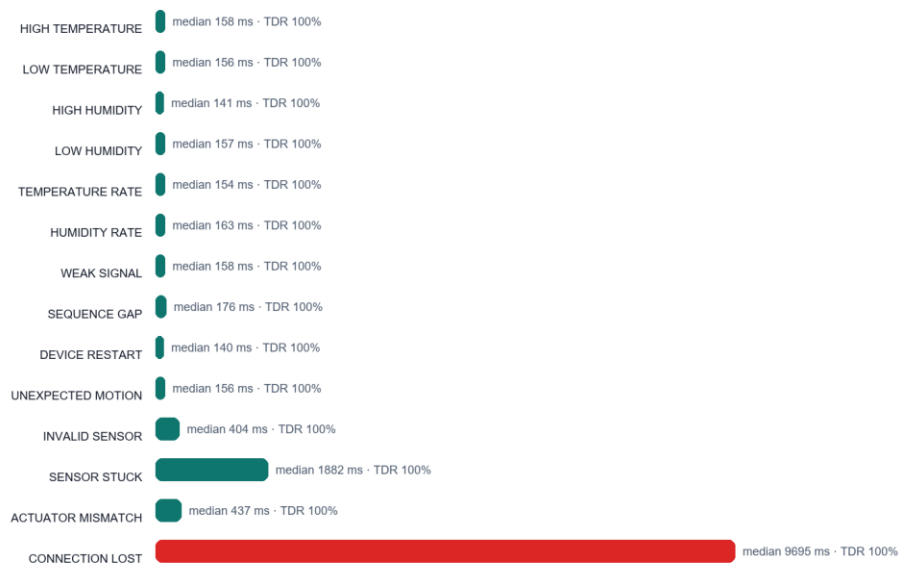
4.1. Phương pháp

Acceptance harness gửi payload qua HTTP Device API để mọi mẫu vẫn đi qua Device Profile, Rule Engine, relation, Asset và Alarm như MQTT. Logical timestamp nằm trong quá khứ để tăng tốc mà không tạo alarm tương lai; lifecycle OFFLINE và stability dùng đồng hồ thật. Mỗi loại bất thường chạy 10 lượt, xen giữa bởi cửa sổ baseline và thao tác clear alarm.

Chỉ số	Cách tính	Ngưỡng
TDR	Lượt có anomaly_type đúng / tổng lượt	$\geq 90\%$
Alarm rate	Lượt có active alarm đúng loại / tổng	Theo dõi
FAR	Baseline bị cảnh báo sai / tổng baseline	$\leq 5\%$
Continuity	POST thành công / POST đã thử	$\geq 95\%$
Sequence consistency	Bước tăng đúng / bước bình thường	$\geq 99\%$
Connection latency	Từ activity cuối đến OFFLINE	≤ 15 s

4.2. Kết quả acceptance

KẾT QUẢ KIỂM THỬ THEO KỊCH BẢN



Thanh biểu diễn median latency; connection-loss dùng timeout lifecycle nên có thang thời gian lớn hơn.

Hình 4.1. Median latency và TDR theo kịch bản

Chỉ số	Kết quả	Đánh giá
Số lượt	140	10 lượt/loại
True Detection Rate	100.00%	PASS
Alarm creation rate	100.00%	Theo dõi
False Alarm Rate	0.00%	PASS
Data continuity	100.00%	PASS
Sequence consistency	100.00%	PASS
Median / p95 latency	172 / 9650 ms	PASS
Connection max	9888 ms	PASS

Kịch bản	n / TDR	Media n ms	Max ms
HIGH_TEMPERATURE	10 / 100%	158	202
LOW_TEMPERATURE	10 / 100%	156	229
HIGH_HUMIDITY	10 / 100%	141	201
LOW_HUMIDITY	10 / 100%	157	199
TEMPERATURE_RATE	10 / 100%	154	173
HUMIDITY_RATE	10 / 100%	163	196
WEAK_SIGNAL	10 / 100%	158	201
SEQUENCE_GAP	10 / 100%	176	197
DEVICE_RESTART	10 / 100%	140	188
UNEXPECTED_MOTION	10 / 100%	156	174
INVALID_SENSOR	10 / 100%	404	462
SENSOR_STUCK	10 / 100%	1882	2088
ACTUATOR_MISMATCH	10 / 100%	437	527
CONNECTION_LOST	10 / 100%	9695	9888

Những lỗi ngưỡng/rate/sequence có độ trễ xử lý Rule Engine thấp; sensor-stuck cần tích lũy 12 mẫu nên lâu hơn. Connection-lost phụ thuộc inactivity timeout và chu kỳ state checker, vì vậy có độ trễ lớn nhất nhưng vẫn phải dưới 15 giây. Sự khác biệt là chủ đích của luật, không phải lỗi hiệu năng.

4.3. Smoke test vận hành liên tục

Thuộc tính	Kết quả
Thời lượng	1.0 phút
Chu kỳ	5.0 giây
Mẫu kỳ vọng / nhận	12 / 12
Continuity	100.00%
NORMAL	100.00%
Online	100.00%
Max latency	146 ms
Kết luận	PASS (smoke test)

Dữ liệu stability dùng sóng sin biên độ nhỏ quanh 22 °C và 65 %RH, không cố định hoàn toàn để tránh sensor-stuck và không đổi quá nhanh để kích hoạt rate anomaly. Mỗi sample dùng timestamp thời gian thật, đợi Asset twin cập nhật và ghi CSV để có thể kiểm tra lại. Theo phạm vi bài tập đã thống nhất, bản nộ chạy smoke test ngắn thay cho phép thử 30 phút; vì vậy kết quả này chứng minh luồng hoạt động liên tục trong khoảng đã đo, không đại diện cho độ bền dài hạn.

4.4. Kiểm thử Wokwi và RPC

- Firmware build thành công cho esp32dev; Wokwi dùng `host.wokwi.internal:1883`.
- Serial cần chứng minh Wi-Fi connected, MQTT connected và publish OK.
- Thay DHT22/PIR phải làm dashboard cập nhật đúng key, không sinh cảm biến giả.
- LED switch gửi setLed; telemetry led_state và physical LED phải đồng nhất.
- Dùng mô phỏng tạo OFFLINE/CONNECTION_LOST; chạy lại tạo RECOVERY.

4.5. Phân tích lỗi trong quá trình hoàn thiện

Một vòng test ban đầu tạo logical timestamp vượt thời gian thật, làm node update alarm báo `Alarm start ts can't be greater then alarm end ts`. Debug failures-only xác định đúng nguyên nhân. Test được sửa để xóa cả future telemetry theo range 1970–3000, chạy accelerated data trong cửa sổ quá khứ và chuyển lifecycle sang wall-clock. Đây là ví dụ về việc phải kiểm thử cả hệ thống giám sát, không chỉ logic nghiệp vụ.

Một race condition sensor-stuck cũng được loại bằng cách chờ `last\_seen` trên Asset sau từng sample, bảo đảm mỗi giá trị đã đi trọn luồng Device→Rule Engine→Asset trước khi gửi mẫu tiếp theo. Các thay đổi này làm bộ test tái lập và không đánh đổi logic sản phẩm.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Đề tài đã tạo một Digital Twin vận hành được từ đầu đến cuối: node cảm biến/mô phỏng, MQTT, mô hình Asset–Device–Relation, trạng thái tổng hợp, state machine, anomaly detection, alarm, dashboard và vòng RPC. Script triển khai tự động cho phép tái tạo mô hình local; source và export không chứa token. Bộ kiểm thử định lượng đáp ứng tiêu chí trong đặc tả và cung cấp CSV/JSON làm bằng chứng.

- Twin có identity cấp kho, trạng thái hiện tại và lịch sử; không chỉ là biểu đồ sensor.
- Expected/observed state cùng command/feedback LED tạo vòng phản hồi hai chiều.
- 13 nhóm bất thường nghiệp vụ/chất lượng dữ liệu/cấu hình cộng connection-lost được kiểm thử 10 lượt mỗi loại.
- Dashboard có cấu trúc tương tự tư duy solution template Smart Irrigation nhưng đúng mạch và nghiệp vụ kho.

5.2. Hạn chế

- Wokwi chưa phản ánh sai số, nhiễu và độ bền của cảm biến thật.
- Ngưỡng demo chưa gắn với từng loại rau quả, giai đoạn chín hoặc thời gian lưu kho.
- Phiên bản CE dùng Rule Engine thay Calculated Fields/Solution Template của PE.
- Một node cảm biến chưa đánh giá xung đột dữ liệu giữa nhiều zone.
- Môi trường local chưa bật TLS/certificate provisioning như production.

5.3. Hướng phát triển

- Mở rộng Asset hierarchy Warehouse→Zone→Rack→Batch và nhiều Device relation.
- Thêm cảm biến cửa, CO₂, ethylene, điện năng và actuator quạt/làm lạnh thật.
- Profile ngưỡng theo nông sản/lô hàng và lịch vận hành.
- Phân tích drift, dự báo hỏng hóc, bảo trì dự đoán và root-cause graph.
- MQTTS, device certificate, RBAC/customer isolation và audit log production.
- Khi có PE license, đóng gói model/dashboard/rule chain thành Solution Template.

PHỤ LỤC A. HƯỚNG DẪN CHẠY NHANH

```
# 1. Khởi động local
cd deployment
docker compose up -d

# 2. Đồng bộ mô hình
$env:TB_API_KEY='<local API key>'
powershell.exe -ExecutionPolicy Bypass `
-File .\thingsboard\setup.ps1

# 3. Build firmware
powershell.exe -ExecutionPolicy Bypass `
-File .\firmware\build.ps1

# 4. Acceptance và smoke stability
python tests\integration_test.py --trials 10 `
--settle-seconds 0.08 --reset-test-data
python tests\stability_test.py --minutes 2 `
--period-seconds 5
```

PHỤ LỤC B. CẤU TRÚC SOURCE

deployment/	Docker Compose
firmware/	ESP32, Wokwi, PlatformIO
thingsboard/	setup.py, rule scripts, sanitized exports
tests/	integration/stability tests + results
docs/	architecture, rules, deployment, demo
report/	report source/output
slides/	slide source/output

PHỤ LỤC C. TÀI LIỆU THAM KHẢO

- [1] [ThingsBoard – Digital Twin Model](#)
- [2] [ThingsBoard – Digital Twin Entities](#)
- [3] [ThingsBoard – Digital Twin Relations](#)
- [4] [ThingsBoard IoT Hub – Smart Irrigation solution template](#)
- [5] [ThingsBoard – MQTT Device API](#)
- [6] [ThingsBoard – MQTT RPC](#)
- [7] [ThingsBoard – Connectivity status](#)
- [8] [ThingsBoard – Rule Engine monitoring](#)
- [9] [Wokwi – ESP32 Wi-Fi](#)
- [10] [Wokwi – PIR motion sensor](#)